PR 1 — Foundations: what the three experiments (E1–E3) test, and why

Bayesian-Reflex implementation (branch `pr/foundations-e1-e3`)

Scope in one sentence. PR 1 validates the scalar Gaussian inference module of the Bayesian-Reflex implementation: it checks that a single unknown number can be tracked with correct uncertainty (E1), updated one observation at a time without losing information (E2a), and that the prediction errors it produces are correctly scaled by that uncertainty (E3).

Not covered. The reflex loop, function-space inference, drift, action, and the paper's look-up-table and ellipsoidal machinery. Those belong to later pull requests.

Role of prediction error in this PR. Prediction error is a *diagnostic only*. The predictive-coding layer reports it; the belief update uses the raw observation and never sees it. Whether the error should drive learning is an open design decision, deferred to experiment E7. This wording applies throughout the document.

1 Why start here

The Bayesian Reflex (arXiv 2605.02825 and 2608.00492) rests on three ideas: maintain a belief, update it sequentially, and act on its uncertainty. Before testing any advanced machinery, the smallest possible version must be provably correct, because every later experiment is compared against an oracle, an exact reference answer. If the smallest version is wrong, nothing built on it can be trusted.

The scalar Gaussian case is ideal for this because the exact answer is known in closed form. E1–E3 are therefore closer to rigorous unit tests of the implementation than to research claims, and the pull request states this explicitly.

2 The model and the code under test

2.1 Model

One unknown number θ with a Gaussian prior, observed through Gaussian noise:

$$\theta \sim \mathcal{N}(\mu_0, s_0^2), \qquad \mu_0 = 0,\ s_0 = 1 \text{ in E1 and E3}, \tag{1}$$

$$y_t = \theta + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma_y^2), \quad \sigma_y = 0.5 \text{ in E1};\ \sigma_y \in \{0.1, 1.0\} \text{ in E3}. \tag{2}$$

2.2 Exact (conjugate) update in precision form

After each observation y_t, with posterior precision $\Lambda_t = 1/\sigma_t^2$:

$$\Lambda_t = \Lambda_{t-1} + \frac{1}{\sigma_y^2}, \tag{3}$$

$$\mu_t = \frac{\Lambda_{t-1}\mu_{t-1} + y_t/\sigma_y^2}{\Lambda_t}, \tag{4}$$

$$\sigma_t^2 = \frac{1}{\Lambda_t}. \tag{5}$$

2.3 Code components

All components live in `src/bayesian_reflex/`.

Component	Role
`GenerativeModel` (interface)	`update(observation)`, `predict()`, `sample(n,rng)`: the contract that every later belief model (GP, PC-FSVI, ...) implements.
`ScalarGaussianBelief`	The update above, vectorised over many independent trials at once. `update` accepts a scalar or a 1-D array of shape `(n_trials,)`; any other shape, and any NaN or infinite value, raises `ValueError` *before* the belief is modified. Constructor parameters are validated the same way.
`PredictiveCodingLayer`	Reports discrepancies only: raw error $e = y - \hat{y}$, predictive precision $1/v$, precision-weighted error e/v, and standardised error $z = e/\sqrt{v}$ (see Section 2.4).
`BayesianReflex`	Orchestrates one step: predict → report error (diagnostic only) → update belief from the raw observation → (optionally) act. The error is not an input to the update; a test checks that the belief after `step` equals the belief after a bare `update`.

Design point. The predictive-coding layer here only reports errors; it does not perform the Bayesian update. Whether the error should drive learning is an explicit open design decision, handled in a later experiment (E7).

2.4 Three different error quantities

Let $e_t = y_t - \hat{y}_t$ and $v_t = \sigma_{t-1}^2 + \sigma_y^2$ (predictive variance, computed *before* the update).

Quantity	Code	Formula	Behaviour when the belief is correct
raw error	`prediction_error`	e_t	scale follows σ_y
precision-weighted error	`precision_weighted_error`	e_t/v_t	variance $1/v_t$; **not** unit variance
standardised error	`standardized_error`	$e_t/\sqrt{v_t}$	$\mathcal{N}(0,1)$ and independent over t

They are related by $z_t = \sqrt{v_t}\,\mathrm{pw}_t$, where $\mathrm{pw}_t = e_t/v_t$. E3's calibration claim ($\mathcal{N}(0,1)$, independent) is about the *standardised* error z_t. The *precision-weighted* error is a different quantity: it is

checked in E3 as an identity and as the quantity that multiplies the prior variance in the Kalman update (Section 5.3), not as a calibrated unit-variance signal.

3 E1 — Is the belief accurate and calibrated?

3.1 Question

Given noisy observations, does the posterior have the right mean and the right width? A belief can be accurate but over-confident. For a Bayesian system, uncertainty is useful only if it is calibrated.

3.2 Method: simulation-based calibration (SBC)

Checking one fixed θ would not test calibration: a fixed truth that falls in the tail of the prior makes intervals under-cover even with correct code. Instead:

1. draw the true $\theta^{(i)} \sim \mathcal{N}(\mu_0, s_0^2)$ for 1000 trials ($\times$ 10 seeds = 10,000 trials);
2. generate $T = 50$ observations for each trial from that $\theta^{(i)}$;
3. update the belief sequentially, one observation at a time;
4. at the checkpoints $t \in \{1, 5, 10, 25, 50\}$, ask where the true $\theta^{(i)}$ falls in the posterior.

If the implementation is correct, the true θ behaves like one more draw from the posterior, so its rank among L posterior samples is uniform:

$$r_i = \#\{\ell \le L : \theta_\ell^{(i)} < \theta^{(i)}\} \sim \text{Uniform}\{0, \dots, L\}. \tag{6}$$

Here $L = 99$ posterior draws per trial are used and ranks are binned into 10 bins.

3.3 Metrics and pre-fixed criteria

Check	Criterion
50% / 80% / 95% interval coverage	within 3 binomial standard errors of nominal (tolerances ± 0.015/ ± 0.012/ ± 0.0065 at 10,000 trials)
Rank uniformity	χ^2 test on the 10-bin rank histogram, $p > 0.001$
Probability-integral transform (PIT)	Kolmogorov–Smirnov test against Uniform$(0, 1)$, $p > 0.001$
Accuracy against claimed uncertainty	RMSE$(\mu_t)/\sqrt{\overline{\sigma_t^2}}$ within 3% of 1 (for a correct posterior $\mathbb{E}[(\mu_t - \theta)^2] = \mathbb{E}[\sigma_t^2]$)
Closed-form sanity	the code's posterior standard deviation equals $(1/s_0^2 + t/\sigma_y^2)^{-1/2}$ to $< 10^{-12}$ for every trial and every t (a reported check; previously a bare `assert`)

3.4 Negative control (must fail)

The same belief, told the wrong noise level (σ_y assumed 0.25 while the data use 0.5). If the test cannot detect this, it proves nothing.

3.5 Secondary result (descriptive, not pass/fail): prior mismatch

With true $\theta = 2.0$, a prior centred at 10 needs 21 observations before the mean bias falls below 0.1, against 5 for a prior centred at 0. Its 95% coverage at $t = 1$ is 0.03. This shows how long a bad prior keeps its influence. This is a single run (500 trials, seed 123), is not pass/fail, and is not covered by the ten-seed rule.

3.6 Result

33 of 33 checks pass (32 calibration checks and controls plus the closed-form check). The 95% coverage is 0.950–0.953 at every checkpoint, and RMSE/sd lies in 0.985–1.009. For the control, the 95% coverage collapses to 0.66 and the KS test rejects ($p \approx 0$).

3.7 Interpretation and limits

For a conjugate Gaussian model, calibration is guaranteed analytically, so passing shows the code is right, not that nature is Gaussian. The same SBC harness is reused later for non-Gaussian methods.

4 E2a — Does sequential updating equal batch updating?

4.1 Question

The central property of recursive Bayes (paper Theorem 2.1): processing observations one by one must give exactly the posterior obtained from all data at once,

$$p(\theta \mid y_{1:t}) \propto p(y_t \mid \theta)\, p(\theta \mid y_{1:t-1}). \tag{7}$$

Any information loss or numerical drift would show up here.

4.2 Method and seeds

E2a follows the ten-seed rule. For each of **10 seeds (0–9)** it draws 100 random problems (random prior mean, prior standard deviation, noise level, and true θ), runs the sequential update for $T = 100$ observations, and compares at every step with a batch oracle: 1000 problems in total. A stress test then runs 20 further problems per seed at $T = 100{,}000$ (200 in total). **Every criterion is evaluated on the worst case over all seeds and problems**, so a single bad seed fails the check; the negative control must differ on every seed (its minimum over seeds is tested).

The oracle is the closed-form posterior of the first t observations,

$$\Lambda_t^{\text{batch}} = \frac{1}{s_0^2} + \frac{t}{\sigma_y^2}, \qquad \mu_t^{\text{batch}} = \frac{\mu_0/s_0^2 + \sum_{k \le t} y_k/\sigma_y^2}{\Lambda_t^{\text{batch}}}, \tag{8}$$

computed in extended precision (`longdouble`), so it is independent of the double-precision recursion being tested.

4.3 Metrics

The absolute difference in mean, the absolute difference in variance, and

$$\mathrm{KL}\big(\mathcal{N}(\mu^{\text{seq}}, \sigma^{2,\text{seq}}) \,\big\|\, \mathcal{N}(\mu^{\text{batch}}, \sigma^{2,\text{batch}})\big), \tag{9}$$

evaluated in a cancellation-safe form (using `log1p`).

4.4 Pre-fixed criteria and observed values (worst case over 10 seeds)

Check	Criterion	Observed
Max difference in mean	$< 10^{-6}$	2.2×10^{-14}
Max difference in variance	$< 10^{-6}$	5.3×10^{-16}
Max KL	$< 10^{-12}$	2.3×10^{-25}
Order independence (shuffled data)	$< 10^{-10}$	8.0×10^{-15}
Negative control: drop every 10th observation	must differ ($> 10^{-3}$) on every seed	2.3×10^{-2} (minimum over seeds)
Stress test $T = 100{,}000$ (200 problems): mean / relative variance / KL	finite, same criteria	2.5×10^{-11} / 2.7×10^{-12} / 1.4×10^{-16}

4.5 Result

9 of 9 checks pass.

4.6 Honest note

The worst-case mean error grows with the number of steps (log–log slope ≈ 1.24 over the 10-seed stress test; the earlier single-seed run gave ≈ 1.06), consistent with accumulated rounding. The slope is a fit to a worst-case envelope and varies with the seeds, so it should not be read as an exact power law. The error stays far below 10^{-6} even at 100,000 steps, but this does not prove stability for arbitrary horizons.

5 E3 — Are prediction errors correctly scaled by the belief's uncertainty?

5.1 Question

E3 is the first predictive-coding quantity. Before seeing y_t, the belief predicts it with mean $\hat{y}_t = \mu_{t-1}$ and predictive variance $v_t = \sigma_{t-1}^2 + \sigma_y^2$, which is parameter uncertainty plus observation noise. The standardised error

$$z_t = \frac{y_t - \hat{y}_t}{\sqrt{\sigma_{t-1}^2 + \sigma_y^2}} \tag{10}$$

should then be exactly $\mathcal{N}(0, 1)$ and independent over time if the belief is correct. This formalises the statement that a 0.5 error means something different when the uncertainty is 0.1 than when it is 2.0.

Important detail. z_t uses the variance from *before* the update. Using the post-update variance is a classic mistake and is tested as a control.

5.2 Setup and scope of each part

θ drawn from the prior; 1000 trials $\times$ 10 seeds (0–9) $\times$ $T = 50$; two noise levels, $\sigma_y \in \{0.1, 1.0\}$. E3 has two parts with different scope:

- **Part A (shared framework).** Everything in Sections 5.3–5.4 uses `ScalarGaussianBelief` and `PredictiveCodingLayer`, and so tests the implementation classes (raw, standardised, and precision-weighted error).
- **Part B (standalone).** The unknown-variance Student-t extension (Section 5.6) is self-contained NumPy/SciPy code (`nig_run`). It does *not* call `ScalarGaussianBelief`, `PredictiveCodingLayer` or `GenerativeModel`. It validates the closed-form NIG predictive derivation and its PIT, **not** the shared framework. Its rows are tagged `[standalone]` in the output and in `results/e3_results.json`.

5.3 Checks, Part A (for each σ_y)

Check	Criterion
Mean of z	within 3 SE of 0
Variance of z	within 3 SE of 1
Lag-1 autocorrelation	within 3 SE of 0 (innovations must be independent)
Distribution	KS test against $\mathcal{N}(0,1)$ at $t = 1, 5, 25, 50$, $p > 0.001$
Kalman identity	belief update equals gain $\times$ raw error, with gain $\sigma^2_{t-1}/(\sigma^2_{t-1} + \sigma^2_y)$, to $< 10^{-12}$ (observed 9×10^{-16})
`precision_weighted_error` identity	$\mathrm{pw}_t \sqrt{v_t} = z_t$ to $< 10^{-12}$ (observed $\leq 2 \times 10^{-15}$)
`precision_weighted_error` scale	$\mathrm{Var}(\mathrm{pw}_t)\, v_t$ within 3 SE of 1 at $t = 1$ and $t = 50$ (i.e. variance $1/v_t$, not 1)
`precision_weighted_error` in the update	$\mu_t = \mu_{t-1} + \sigma^2_{t-1}\, \mathrm{pw}_t$ to $< 10^{-12}$ (observed 9×10^{-16})

Explicitly, the Kalman identity being checked is

$$\mu_t = \mu_{t-1} + \frac{\sigma^2_{t-1}}{\sigma^2_{t-1} + \sigma^2_y}\,(y_t - \mu_{t-1}) = \mu_{t-1} + \sigma^2_{t-1}\,\underbrace{\frac{y_t - \mu_{t-1}}{v_t}}_{\mathrm{pw}_t}, \tag{11}$$

where the second form is exactly the output of `precision_weighted_error()`. This is how E3 tests that function directly. It is an identity about the *relationship* between the error signal and the update; it does not mean the error drives learning in the implementation (the belief update does not call the predictive-coding layer).

5.4 Precision weighting made visible

The spread of the raw error follows σ_y (ratio 10.0 between $\sigma_y = 1.0$ and $\sigma_y = 0.1$), while the spread of z does not (ratio 1.000). The standardised error removes the noise scale, as intended. The precision-weighted error does *not* do this: its variance is $1/v_t$ (about 96 at $t = 50$ for $\sigma_y = 0.1$, about 0.5 at $t = 1$ for $\sigma_y = 1.0$), which is why it is a different quantity from z_t.

5.5 Negative controls (must fail)

- Wrong σ_y (0.5× the true value): $\mathrm{Var}(z) \approx 3.9$ instead of 1.
- Stale variance (post-update instead of pre-update): $\mathrm{Var}(z)$ over the first five steps is 11.0 for $\sigma_y = 0.1$ and 1.12 for $\sigma_y = 1.0$, and the KS test rejects at $t = 1$.
- Raw (unweighted) error used in the update, $\mu_{t-1} + \sigma^2_{t-1} e_t$: deviates from the true update by 0.26 ($\sigma_y = 0.1$) and 2.7 ($\sigma_y = 1.0$), so the precision-weighted identity check has power.
- Precision-weighted error treated as unit variance: $\max_t |\mathrm{Var}(\mathrm{pw}_t) - 1|$ is 96 ($\sigma_y = 0.1$) and 0.51 ($\sigma_y = 1.0$).

5.6 Extension, Part B (paper §5.2, standalone): unknown noise variance

With a Normal–Inverse-Gamma prior the predictive distribution is a Student-t. With hyperparameters (m, κ, a, b) after $t-1$ observations,

$$y_t \mid y_{1:t-1} \sim t_{2a_{t-1}}\left(m_{t-1},\ \frac{b_{t-1}(\kappa_{t-1}+1)}{a_{t-1}\kappa_{t-1}}\right), \tag{12}$$

so z_t should follow a t distribution with $2a_{t-1}$ degrees of freedom, not a Gaussian. The PIT through the t CDF passes ($p = 0.06$–0.79); using the Gaussian CDF instead is rejected ($p \approx 6 \times 10^{-24}$), which is the control.

Scope. This part checks the closed-form NIG derivation and its PIT in standalone code. It shares no code with the implementation classes, so it says nothing about whether `ScalarGaussianBelief`, `PredictiveCodingLayer` or `GenerativeModel` are correct in the unknown-variance setting. Validating the framework there would need a NIG belief implementing `GenerativeModel`, which is future work.

5.7 Result

41 of 41 checks pass (29 original checks plus 12 for `precision_weighted_error` and its controls).

5.8 Sensitivity reruns on fresh seeds

In the primary run two statistics at $\sigma_y = 0.1$ (the mean of z and the lag-1 autocorrelation) sat close to their limits. The same checks, with unchanged criteria, were rerun on two fresh sets of 10 seeds that the primary run never used (rerun A: seeds 10–19, rerun B: seeds 20–29), for *both* σ_y. The command is `pythonexperiments/e3_rerun_sensitivity.py`; seeds and summaries are in `results/e3_rerun_sensitivity.json`.

σ_y	seed set	mean z (limit 0.0042)	$\mathrm{Var}(z)$ (limit 1 ± 0.0060)	lag-1 (limit 0.0042)
0.1	0–9 (primary)	0.0028	0.9969	−0.0037
0.1	10–19 (A)	−0.0036	0.9996	−0.0001
0.1	20–29 (B)	−0.0018	0.9966	−0.0008
1.0	0–9 (primary)	0.0013	0.9969	−0.0037
1.0	10–19 (A)	−0.0039	0.9996	−0.0003
1.0	20–29 (B)	−0.0015	0.9967	−0.0009

All statistics are inside their pre-fixed limits. Two caveats. First, $\sigma_y = 0.1$ and $\sigma_y = 1.0$ reuse the same seeds, so the two rows for a seed set are not independent evidence. Second, several

mean-z values lie at roughly 2–2.8 standard errors: inside the 3-SE rule, but not far inside. This is consistent with chance; it does not prove it.

Correction to an earlier statement. A previous version of this description said only that $\sigma_y = 1.0$ was rerun on two fresh sets of 10 seeds. That does not test the $\sigma_y = 0.1$ statistics that were near their limits, and the seeds were not recorded. The table above, produced by a committed script, replaces it and covers both noise levels.

5.9 Caveats

The experiment does not show that the predictive-coding layer drives learning; it only shows that its error signal is correctly standardised, and that the precision-weighted error obeys the stated identities.

6 What PR 1 claims, and what it does not

Claims (exactly these):

1. The scalar Gaussian belief is accurate and calibrated (E1).
2. Sequential updating reproduces the batch posterior to numerical precision on 10 seeds, including a 100,000-step stress test (E2a).
3. One-step-ahead prediction errors from the shared classes are correctly standardised by the Bayesian predictive, and the precision-weighted error obeys its stated identities (E3, Part A). Separately, the closed-form Student-t predictive for unknown variance is correct in standalone code (E3, Part B).

Does not claim: anything about function-space belief, drift, uncertainty-driven action, the closed loop, the paper-specific look-up-table and ellipsoidal sampler, that prediction error drives learning, or that the shared framework is validated for the unknown-variance case.

Method rules (carried through all later pull requests):

- pass/fail criteria are fixed before running;
- every positive test has a negative control that must fail;
- at least 10 seeds for every pass/fail result, with tolerances derived from confidence intervals (descriptive results, such as the E1 prior-mismatch run, are labelled as such);
- every experiment command exits with status 0 only if all required checks pass, and 1 otherwise;
- claims are limited to what a stage actually tests.

7 Reproduce

```
pip install -r requirements.txt
pytest                                          # 27 passed
python experiments/e1_scalar_belief.py          # exit 0, "...foundation only): PASS"
python experiments/e2_sequential_equivalence.py # exit 0, "...vs batch oracle): PASS"
python experiments/e3_prediction_error.py       # exit 0, "...prediction error): PASS"
python experiments/e3_rerun_sensitivity.py      # exit 0, "...both sigma_y): PASS"
```